

TP Grupal: Money Laundering Analysis

Post-Mortem

Facultad de Ingeniería de la Universidad de Buenos Aires

Sistemas Distribuidos

Cátedra Roca



Integrantes:

Alumno	Padrón
Fernandez, Ignacio Agustín	111019
Molina, Taiel Alexis	109458
Serra, Luciano	110694

Post-mortem

Por qué la robustez obliga a conocer toda la infraestructura

Una lección del desarrollo es que la robustez no se puede agregar mirando un nodo aislado. Para decidir cuándo hacer `ack`, cuándo persistir, qué contar en un EOF y qué hacer ante una caída, hay que conocer RabbitMQ, el protocolo TCP con el cliente, el sharding, la persistencia local, la forma en que cada etapa emite resultados y la semántica de cada query.

Esto hizo que conceptos aparentemente locales, como “un filtro stateless”, terminaran dependiendo de infraestructura compartida. El filtro no acumula negocio, pero necesita conocer duplicados, EOFs, aborts, cantidades emitidas por shard y checkpoints. Sin ese estado mínimo, una caída entre procesar y confirmar podría duplicar procesamiento o dejar al siguiente nodo esperando un EOF con totales incorrectos.

Alternativas no implementadas

Una alternativa evaluada para no depender del affinity que finalmente no fue implementada por por motivos de tiempo fue el uso de un nodo dedicado por cluster que ofrezca el control de secuencia comunicado via RPC. Entre las ventajas que trae esa implementación respecto a la finalmente realizada se pueden destacar:

- Reducción de affinity mediante el uso de competing consumers.
- Desacople de la infraestructura. Los nodos de un cluster podrían contar con menos información de sus nodos previos o siguientes, delegando esa responsabilidad al middleware.

Como contraparte, esta otra implementación genera casos de estado inconsistente (caídas durante el procesamiento del RPC por ejemplo) que requerían ser detectadas y manejadas.

Nodos stateless con estado

La denominación stateless fue útil para diseñar el DAG, pero incompleta para implementar tolerancia a fallos. En la práctica, los nodos stateless son stateless sólo respecto de la transformación de dominio. Operacionalmente tienen estado: necesitan recordar qué `seq` ya vieron, cuántos mensajes emitieron y si una sesión fue abortada. Esta fue una de las principales diferencias entre el diseño ideal y la implementación robusta. Esto implica mayor carga de trabajo en nodos que podrían no tenerla generando cuellos de botella que con un mejor diseño podrían ser salvados.

Cliente caído y reanudación

Ante caídas del gateway, o cortes de conexión donde el cliente conserva su estado local, el sistema permite reanudar la sesión dentro de una ventana de tiempo. Durante el handshake, el cliente vuelve a enviar su `sessionId`; si el gateway reconoce esa sesión, recupera la fase en curso y el último `confirmedSeq`. A partir de esa respuesta, el cliente puede continuar enviando desde el punto confirmado, evitando reiniciar todo el procesamiento.

Ante caídas del cliente, en cambio, no se cuenta con un mecanismo que garantice un estado consistente entre lo que el cliente persiste y lo que el gateway cree que el cliente ya conoce. Para resolver ese caso de manera segura, se optó por abortar la sesión inmediatamente: el gateway marca al cliente como abortado, limpia su estado asociado y propaga un mensaje `abort` hacia los nodos del pipeline para que también eliminen el estado parcial de ese cliente.